

AURA Shop — Complete Testing Guide

This guide covers every shop feature implemented across P0, P1, and P2. Follow the steps in order. Each section lists what to test and what to expect.

0. Prerequisites — Start the environment

0.1 Database & migrations

```
cd backend

# Run all pending migrations (creates shop tables, money decimal columns,
# tracking fields, shipping rates, coupons)
npx prisma migrate deploy

# If you want a fresh database with seed data:
npx prisma migrate reset --force
npm run seed:shop
```

0.2 Start the servers

From the project root:

```
npm run dev
```

This starts:

- Backend on `http://localhost:5000` (or the port in `backend/.env`)
- Frontend on `http://localhost:3000`

0.3 Admin access

You need an admin account. If you don't have one:

```
cd backend
npx prisma studio
```

In Prisma Studio, open the `User` table, find your user, and set `role` to `ADMIN` . Or create a new user with `role: ADMIN` .

Log in on the frontend at <http://localhost:3000/login> .

0.4 Seed shop data (if not already seeded)

```
cd backend  
npm run seed:shop
```

This creates sample categories, products (with variants), and images.

1. Catalog Browsing (P0/P1)

1.1 Product listing

- Go to <http://localhost:3000/shop>
- **Expect:** Product grid loads with images, names, prices
- **Expect:** Categories sidebar/filters appear

1.2 Search

- Type a product name in the search box
- **Expect:** Results filter to matching products

1.3 Price filter

- Set min and/or max price
- **Expect:** Products filter by price range
- **Test combined:** Type a search term AND set a price range
- **Expect:** Both filters apply together (P1-9 fix — previously search was lost)

1.4 Price sorting

- Sort by "Price: Low to High"
- **Expect:** Products ordered by effective price (salePrice ?? basePrice)
- Sort by "Price: High to Low"
- **Expect:** Reverse order

1.5 Category filter

- Click a category
- **Expect:** Only products in that category appear
- **Expect:** URL contains the category slug

1.6 Product detail page

- Click any product
- **Expect:** Full product page with images, description, price, variants
- **Expect:** Browser tab title shows product name (P2-6 SEO)
- **Expect:** View page source / inspect `<meta>` tags — `og:title` , `og:description` , `og:image` , `canonical` link present

1.7 Out-of-stock products

- A product with `status: OUT_OF_STOCK` should not appear in the catalog
 - A product with `status: ARCHIVED` should not appear
 - A product with `status: DRAFT` should not appear
-

2. Cart (P0)

2.1 Add to cart

- On a product detail page, select a variant (if applicable), choose quantity, click "Add to Cart"
- **Expect:** Cart badge/counter updates
- **Expect:** Item persists after page refresh (localStorage `aura-shop-cart`)

2.2 Cart management

- Go to cart page
- **Expect:** Items listed with image, name, variant, price, quantity
- Change quantity
- **Expect:** Subtotal recalculates
- Remove an item
- **Expect:** Item disappears, subtotal updates

2.3 Empty cart

- Remove all items
 - **Expect:** "Your cart is empty" message with link to shop
-

3. Checkout — Guest (P0/P1/P2)

3.1 Guest checkout with bank transfer

1. Add items to cart, go to checkout
2. Fill in shipping details (name, phone, region, city, address)
3. Select "Bank Transfer" as payment method
4. **Expect:** Receipt upload field appears (required for bank transfer)
5. Upload a receipt image (JPG/PNG)
6. **Expect:** Filename shown next to upload button
7. Click "Place Order"
8. **Expect:** Redirects to order confirmation page
9. **Expect:** URL contains `?guestToken=...` (fresh per-order token)
10. **Expect:** Order number displayed (format: `AURA-YYYY-XXXXX`)

3.2 Guest checkout with COD

1. Repeat above but select "Cash on Delivery"
2. **Expect:** No receipt upload required
3. Place order without uploading anything
4. **Expect:** Order succeeds

3.3 Receipt enforcement (P0-6)

1. Select "Bank Transfer" or "Mobile Money"
2. Try to place order WITHOUT uploading a receipt
3. **Expect:** Error message "A payment receipt is required..."

3.4 Receipt file validation (P1-15)

1. Upload a non-image file (e.g., a `.txt` or `.zip` renamed to `.jpg`)
2. Place order
3. **Expect:** Error "does not appear to be a valid image or PDF file"

3.5 Idempotency (P1-11)

1. Open browser DevTools → Network tab
2. Place an order
3. In the Network tab, find the `POST /api/shop/orders` request
4. Right-click → "Copy as fetch"
5. Run the same fetch in the console
6. **Expect:** The second request returns the SAME order (not a duplicate)

7. Check the admin orders — **expect only 1 new order, not 2**

3.6 Shipping cost (P2-3)

1. On checkout, enter a region (e.g., "Addis Ababa")
2. **Expect:** Shipping cost updates in the order summary after ~0.5s
3. If shipping rates are configured, **expect:** non-zero shipping cost
4. If no rates configured, **expect:** "Free" shipping

3.7 Coupon application (P2-8)

1. First, create a coupon in admin (see section 8.5 below)
2. On checkout, enter the coupon code and click "Apply"
3. **Expect:** Discount line appears in order summary
4. **Expect:** Total recalculates with discount
5. Try an invalid code
6. **Expect:** Error message "Invalid coupon code"
7. Try an expired coupon
8. **Expect:** Error "This coupon has expired"

3.8 Multiple guest orders (P0-2)

1. Place one guest order
2. Go back to shop, add items, place another guest order
3. **Expect:** Both orders succeed (no "duplicate guest token" error)
4. **Expect:** Each order has a different `guestToken` in the URL

4. Checkout — Authenticated User (P0)

4.1 Authenticated checkout

1. Log in
2. Add items, go to checkout
3. **Expect:** Shipping name pre-filled from profile
4. Place order
5. **Expect:** Order is created with `userId` set
6. Go to "My Orders" (`/shop/orders`)
7. **Expect:** The order appears in the list

4.2 Order ownership

1. Log in as User A, place an order
2. Log in as User B
3. Try to access User A's order by URL: `/shop/orders/<orderA_id>`
4. **Expect:** 403 "Not authorized" error

4.3 Guest cannot access authenticated order

1. Log in, place an order (no guestToken in URL)
 2. Open the order URL in an incognito window (no auth, no guestToken)
 3. **Expect:** 403 error
-

5. Order Viewing & Recovery (P2-5)

5.1 My Orders (authenticated)

1. Log in, go to `/shop/orders`
2. **Expect:** List of your orders with status badges, totals, dates
3. Click an order
4. **Expect:** Full order detail (items, shipping address, status history)

5.2 Guest order recovery

1. Place a guest order, note the order number and phone used
2. Open a new browser/incognito window
3. Go to `/shop/orders`
4. **Expect:** "Track your order" form (guest view, no orders list)
5. Enter the order number and phone
6. Click "Find Order"
7. **Expect:** Redirects to the order confirmation page

5.3 Wrong phone

1. Use the lookup form with the correct order number but wrong phone
 2. **Expect:** "Order not found" error
-

6. Wishlist (P2-4)

6.1 Add to wishlist

1. Log in
2. Go to a product detail page
3. Click the heart icon
4. **Expect:** Heart fills in red
5. Refresh the page
6. **Expect:** Heart stays red (persisted server-side)

6.2 Remove from wishlist

1. Click the filled heart
2. **Expect:** Heart returns to outline state

6.3 Guest cannot use wishlist

1. Log out
2. Go to a product detail page
3. **Expect:** Heart button is disabled/greyed out
4. **Expect:** Tooltip says "Log in to use wishlist"

7. Admin — Product Management

7.1 Create a product

1. Log in as admin, go to `/admin/shop`
2. Go to Products tab
3. Click "Add Product"
4. Fill in: name, description, category, base price, sale price (optional), SKU, stock
5. Save
6. **Expect:** Product appears in list
7. **Expect:** Slug auto-generated from name

7.2 Product with variants

1. Create a product
2. Add variants (size, color, price delta, stock)
3. **Expect:** Variants appear in product detail on the storefront

7.3 Upload product images

1. Edit a product
2. Upload an image (JPG/PNG/WebP)
3. **Expect:** Image appears in product gallery
4. Upload a non-image file (e.g., `.txt` renamed to `.jpg`)
5. **Expect:** Error "does not appear to be a valid image"

7.4 Delete product image (P1-15)

1. Delete a product image in admin
2. **Expect:** Image removed from product
3. **Expect:** Supabase storage object also deleted (check Supabase dashboard)

7.5 Category management (P1-13)

1. Create a category "Yoga Mats" (slug auto-generated as `yoga-mats`)
2. Rename it to "Premium Yoga Mats"
3. **Expect:** Name updates but slug stays `yoga-mats` (immutable on rename)
4. Verify the category URL still works: `/shop?category=yoga-mats`

7.6 Stock management

1. Set a product's stock to 5
2. On storefront, add 6 to cart
3. **Expect:** Error or stock limit on checkout
4. Set stock to 0
5. **Expect:** Product auto-transitions to `OUT_OF_STOCK` status
6. Set stock back to 10
7. **Expect:** Product auto-transitions back to `ACTIVE`

8. Admin — Order Management

8.1 View orders

1. Go to `/admin/shop` → Orders tab
2. **Expect:** All orders listed with order number, customer, status, payment status, total
3. Filter by status (e.g., "Pending")
4. **Expect:** Only pending orders shown
5. Search by order number or customer name

8.2 View order detail

1. Click "View" on an order
2. **Expect:** Modal with customer info, shipping address, items, totals
3. **Expect:** Status history timeline
4. **Expect:** Payment receipt link (if uploaded)

8.3 Verify payment

1. Find an order with `paymentStatus: PENDING` (bank transfer/mobile money)
2. Click "Verify Payment"
3. **Expect:** Payment status changes to VERIFIED
4. **Expect:** Order status changes to CONFIRMED
5. If SMTP is configured, **expect:** Customer receives "Payment Verified" email
6. Check server console for email log (if SMTP not configured)

8.4 Reject payment

1. Find another pending payment order
2. Click "Reject Payment"
3. **Expect:** Payment status changes to REJECTED

8.5 Update order status

1. Select a new status from the dropdown (e.g., "Processing")
2. **Expect:** Status updates, new entry in status history
3. Set status to "Shipped"
4. If SMTP configured, **expect:** Customer receives "Order Shipped" email
5. Set status to "Delivered"
6. If SMTP configured, **expect:** Customer receives "Order Delivered" email

8.6 Tracking details (P2-2)

1. Open order detail modal
2. Fill in Carrier (e.g., "Ethiopian Postal") and Tracking Number
3. Click "Save Details"
4. **Expect:** Details saved
5. Set status to "Shipped"
6. **Expect:** Email (if configured) includes carrier and tracking number

8.7 Adjust shipping cost (P2-2)

1. Open order detail

2. Change "Shipping Cost" from 0 to 200
3. Click "Save Details"
4. **Expect:** Total recalculates (subtotal + shipping - discount)

8.8 Cancel order & restock (P0-3/P1-14)

1. Find a PENDING order that has items with tracked stock
 2. Note the current stock of those products (check admin product list)
 3. Set order status to "Cancelled"
 4. **Expect:** Stock is restocked (incremented back)
 5. Verify in admin product list — stock should be higher than before
 6. Now try to set the same cancelled order to "Refunded"
 7. **Expect:** Status changes but stock is NOT restocked again (double-restock guard)
-

9. Admin — Shipping Rates (P2-3)

9.1 Create shipping rates

1. Go to admin shipping rates section
2. Create a rate:
 - Region: "Addis Ababa"
 - Rate: 100
 - Free shipping over: 5000
3. Create a default rate:
 - Region: (leave blank)
 - Rate: 250
4. **Expect:** Both rates appear in the list

9.2 Test shipping calculation

1. On storefront checkout, enter region "Addis Ababa" with subtotal < 5000
2. **Expect:** Shipping = 100 ETB
3. Add more items so subtotal >= 5000
4. **Expect:** Shipping = Free
5. Enter region "Oromia" (no specific rate)
6. **Expect:** Shipping = 250 ETB (default rate)

9.3 Deactivate a rate

1. Set the "Addis Ababa" rate to inactive

2. On checkout, enter "Addis Ababa"
 3. **Expect:** Falls back to default rate (250)
-

10. Admin — Coupons (P2-8)

10.1 Create a percentage coupon

1. Go to admin coupons section
2. Create coupon:
 - Code: `SAVE10`
 - Type: PERCENTAGE
 - Value: 10
 - Min subtotal: 1000
 - Max discount: 500
3. **Expect:** Coupon appears in list

10.2 Create a fixed coupon

1. Create coupon:
 - Code: `FLAT200`
 - Type: FIXED
 - Value: 200
2. **Expect:** Coupon appears

10.3 Test coupon on checkout

1. Add items with subtotal ≥ 1000
2. Apply `SAVE10`
3. **Expect:** 10% discount applied (capped at 500)
4. Remove coupon, apply `FLAT200`
5. **Expect:** 200 ETB discount

10.4 Coupon validation

1. Apply `SAVE10` with subtotal < 1000
2. **Expect:** Error "Minimum order of ETB 1,000 required"
3. Create a coupon with `maxUses: 1`
4. Use it once
5. Try to use it again
6. **Expect:** Error "has reached its usage limit"

10.5 Expired coupon

1. Create a coupon with `endsAt` in the past
 2. Try to apply it
 3. **Expect:** Error "This coupon has expired"
-

11. Admin — Analytics (P2-9)

11.1 View analytics summary

1. Go to admin analytics section
2. **Expect:** Summary cards showing:
 - Total orders
 - Pending/confirmed/shipped/delivered/cancelled counts
 - Total revenue (verified payments)
 - AOV (average order value)
 - Verified orders count
3. **Expect:** Top 5 products by revenue
4. **Expect:** Revenue over time chart (last 30 days, daily breakdown)

11.2 Date filtering

1. Set a start/end date range
 2. **Expect:** Analytics recalculate for that period
-

12. Admin — Low Stock Report (P2-10)

12.1 View low-stock products

1. Go to admin inventory/low-stock section
2. **Expect:** Products with stock $\leq$ threshold (default 5)
3. **Expect:** Variants with stock $\leq$ threshold
4. Change threshold (e.g., `?threshold=20`)
5. **Expect:** More items appear

12.2 Auto OUT_OF_STOCK transition

1. Set a product's stock to 1
2. On storefront, buy 1 unit

3. **Expect:** Product stock becomes 0
4. **Expect:** Product status auto-changes to `OUT_OF_STOCK`
5. **Expect:** Product no longer visible in storefront catalog

12.3 Auto-reactivate on restock

1. In admin, set the `OUT_OF_STOCK` product's stock to 10
 2. **Expect:** Product status auto-changes back to `ACTIVE`
 3. **Expect:** Product visible in storefront again
-

13. Sitemap & SEO (P2-6)

13.1 Sitemap

1. Visit `http://localhost:5000/api/sitemap.xml`
2. **Expect:** Valid XML with URLs for:
 - Static pages (`/` , `/classes` , `/packages` , `/shop` , `/contact`)
 - Class pages
 - Product pages (`/shop/products/<slug>`)
 - Category pages (`/shop?category=<slug>`)

13.2 Product SEO

1. Visit a product detail page
 2. Right-click → "View Page Source"
 3. **Expect:** `<title>` tag includes product name
 4. **Expect:** `<meta name="description">` with product description
 5. **Expect:** `<meta property="og:title">` and `og:description`
 6. **Expect:** `<meta property="og:image">` with product image URL
 7. **Expect:** `<link rel="canonical">` pointing to the product URL
-

14. Rate Limiting (P1-10)

14.1 Catalog rate limit

```
# Send 70 rapid requests to catalog (limit is 60/min)
for i in $(seq 1 70); do
  curl -s -o /dev/null -w "%{http_code}\n"
  http://localhost:5000/api/shop/products
done
```

- Expect: First 60 return 200, then 429 (Too Many Requests)

14.2 Order creation rate limit

- Place 6 orders rapidly (limit is 5 per 15 min)
- Expect: 6th order returns 429

15. API Testing with curl (optional)

15.1 Get products

```
curl http://localhost:5000/api/shop/products
```

15.2 Get product by slug

```
curl http://localhost:5000/api/shop/products/<slug>
```

15.3 Get categories

```
curl http://localhost:5000/api/shop/categories
```

15.4 Shipping quote

```
curl "http://localhost:5000/api/shop/shipping/quote?
region=Addis%20Ababa&subtotal=3000"
```

15.5 Validate coupon

```
curl -X POST http://localhost:5000/api/shop/coupons/validate \  
  -H "Content-Type: application/json" \  
  -d '{"code":"SAVE10","subtotal":3000}'
```

15.6 Guest order lookup

```
curl -X POST http://localhost:5000/api/shop/orders/lookup \  
  -H "Content-Type: application/json" \  
  -d '{"orderNumber":"AURA-2026-00001","phone":"0912345678}"'
```

15.7 Create order (with auth token)

```
curl -X POST http://localhost:5000/api/shop/orders \  
  -H "Authorization: Bearer <your-token>" \  
  -H "Idempotency-Key: $(uuidgen)" \  
  -F 'items=[{"productId":"<id>","quantity":1}]' \  
  -F 'paymentMethod=CASH_ON_DELIVERY' \  
  -F 'shippingFullName=Test User' \  
  -F 'shippingPhone=0912345678' \  
  -F 'shippingRegion=Addis Ababa' \  
  -F 'shippingCity=Addis Ababa' \  
  -F 'shippingAddress=Bole Road 123'
```

15.8 Admin: verify payment

```
curl -X PATCH \  
http://localhost:5000/api/admin/shop/orders/<orderId>/payment \  
  -H "Authorization: Bearer <admin-token>" \  
  -H "Content-Type: application/json" \  
  -d '{"paymentStatus":"VERIFIED}"'
```

15.9 Admin: low-stock report

```
curl http://localhost:5000/api/admin/shop/inventory/low-stock?threshold=5 \  
  -H "Authorization: Bearer <admin-token>"
```

16. Edge Cases & Negative Tests

16.1 Insufficient stock

1. Set a product's stock to 2
2. Try to checkout with quantity 3
3. **Expect:** Error "Insufficient stock for "

16.2 Concurrent orders (race condition)

```
# Run two simultaneous checkouts for the same product with stock=1
# Both should not succeed – only one order goes through
curl -X POST http://localhost:5000/api/shop/orders -d '...' &
curl -X POST http://localhost:5000/api/shop/orders -d '...' &
wait
```

- **Expect:** One succeeds, the other gets "Insufficient stock" error
- **Expect:** Stock never goes negative

16.3 Order number uniqueness

1. Place 10 orders rapidly
2. **Expect:** All 10 have unique order numbers (no collisions)

16.4 Inactive product

1. Set a product to `status: DRAFT`
2. Try to checkout with that product's ID
3. **Expect:** Error "Product not available"

16.5 Inactive variant

1. Set a variant to `isActive: false`
2. Try to checkout with that variant ID
3. **Expect:** Error "Variant not available"

16.6 Deleted product image cleanup

1. Upload an image to a product
2. Note the Supabase URL
3. Delete the image in admin
4. Visit the Supabase URL in a browser
5. **Expect:** 404 or "not found" (object deleted from storage)

Quick Test Checklist

Use this as a fast smoke test:

- ☐ Shop page loads with products
- ☐ Search + price filter work together
- ☐ Product detail page loads with SEO meta tags
- ☐ Add to cart works
- ☐ Guest checkout (COD) succeeds
- ☐ Guest checkout (bank transfer) requires receipt
- ☐ Guest can place multiple orders
- ☐ Authenticated user's orders appear in "My Orders"
- ☐ Guest can recover order via order number + phone
- ☐ Wishlist heart toggles (logged in only)
- ☐ Admin can verify/reject payment
- ☐ Admin can update order status
- ☐ Admin can set tracking number + carrier
- ☐ Admin can cancel order → stock restocked
- ☐ Cancelled → Refunded does NOT double-restock
- ☐ Shipping cost calculates by region
- ☐ Coupon applies discount at checkout
- ☐ Low-stock report shows products $\leq$ threshold
- ☐ OUT_OF_STOCK auto-transitions when stock hits 0
- ☐ Sitemap includes shop product/category URLs
- ☐ Rate limiting blocks excessive requests